

IDS-SQL

príprava na semestrálnu skúšku 2018/2019

V tomto dokumente sú spomenuté len úlohy a riešenia oficiálne poskytnuté cez dataprojektor po skúške. (Je to teda len prepísaná fotka plátna za účelom väčšieho komfortu pri učení.)

1. Predpokladajte časť relačnej databázy leteckej spoločnosti, ktorá obsahuje tabuľky *Cestujuci*(cisloC, meno, mesto), *Let*(cisloL, odkial, kam, datum, cas) a *Letenka*(cisloC, cisloL, miesto, trieda). U cestujúceho je uvedené jeho jednoznačné číslo, meno a mesto bydliska, pri lete je uvedené jednoznačné číslo, odkiaľ a kam sa letí a dátum a čas odletu. V tabuľke *Letenka* je číslo cestujúceho a číslo letu, ktorého sa cestujúci účastní, ďalej miesto v lietadle a trieda. Primárne kľúče tabuliek sú podčiarknuté. Napíšte v jazyku SQL nasledujúce príkazy:

- Napíšte dotaz: Do ktorých destinácií bolo vystavených viac leteniek než do Madridu? (kam, pocet_leteniek)

```
SELECT kam, COUNT(*) AS pocet_leteniek
FROM Let Lt JOIN Letenka La ON Lt.cisloL = La.cisloL
GROUP BY kam
HAVING COUNT(*) > (
    SELECT COUNT(*)
    FROM Let Lt JOIN Letenka La ON Lt.cisloL = La.cisloL
    WHERE kam = 'Madrid')
```

//rôzne varianty zápisu spojenia

//použitie kam <> 'Madrid' v dotaze vyššej úrovne nie je chyba

- Napíšte príkaz pre vytvorenie databázového pohľadu obsahujúceho informácie o cestujúcich, ktorí v roku 2018 aspoň raz leteli z Prahy (cisloC, meno, mesto). Ďalej uveďte, aký význam by malo pridanie klauzule WITH CHECK OPTION k definícii tohto pohľadu a tvrdenie zdôvodnite.

```
CREATE VIEW pohlad AS
SELECT cisloC, meno, mesto
FROM Cestujuci C JOIN Letenka La ON C.cisloC = La.CisloC JOIN Let Lt
ON La.cisloL = Lt.cisloL
WHERE odkial='Praha' AND datum BETWEEN '2018-01-01' AND '2018-12-31'
```

//rôzne varianty zápisu spojenia

- Predpokladajte, že v tabuľke Let je pridaný atribút pocet_cestujucich. Napíšte príkaz pre vytvorenie databázového triggera, ktorý v prípade vloženia nového záznamu do tabuľky *Letenka* zvýši o jedničku hodnotu atribútu pocet_cestujucich.

```

CREATE TRIGGER trigger1
AFTER INSERT ON Letenka
FOR EACH ROW
BEGIN
    UPDATE Let
    SET pocet_cestujucich = pocet_cestujucich + 1
    WHERE cisloL = :NEW.cisloL;
END;

```

//možné použiť aj BEFORE

- Napíšte dotaz: Ktorí cestujúci lietajú **len** z Prahy do Londýna? (cisloC, meno)

```

SELECT cisloC, meno
FROM Cestujuci C JOIN Letenka La ON C.cisloC = La.cisloC JOIN Let Lt
ON La.cisloL = Lt.cisloL
WHERE odkial = 'Praha' AND kam = 'Londyn' AND NOT EXISTS (
    SELECT * FROM Letenka La JOIN Let Lt ON La.cisloL = Lt.cisloL
    WHERE C.cisloC = La.cisloC AND (odkial<>'Praha' OR kam<>'Londyn')
)

```

//rôzne varianty zápisu spojenia

//NOT IN

- Napíšte príkaz pre vytvorenie databázového pohľadu obsahujúceho informácie o počtoch predaných leteniek do business triedy pre jednotlivé linky (odkial, kam, pocet leteniek). Linka zahŕňa všetky lety medzi dvoma rovnakými letiskami (napr. Praha-Londýn). Ďalej uveďte, aký význam by malo pridanie klauzule WITH CHECK OPTION k definícii tohto pohľadu a tvrdenie odôvodnite.

```

CREATE VIEW pohlad AS
SELECT odkial, kam, COUNT (*) AS pocet_leteniek
FROM Let Lt JOIN Letenka La ON Lt.cisloL = La.cisloL
WHERE trieda = 'business'
GROUP BY odkial, kam

```

//rôzne varianty zápisu spojenia

- Predpokladajte, že v tabuľke Let je pridaný atribút pocet volnych miest. Napíšte príkaz pre vytvorenie databázového triggeru, ktorý v prípade zmazania záznamu z tabuľky *Letenka* zvýši o jedničku hodnotu atribútu pocet volnych miest.

```

CREATE TRIGGER trigger1
AFTER DELETE ON Letenka
FOR EACH ROW
BEGIN
    UPDATE Let

```

```

SET pocet_volnych_miest = pocet_volnych_miest + 1
WHERE cisloL = :OLD.cisloL;
END;

```

//možné použiť aj BEFORE

2. Predpokladajte časť relačnej databázy oddelenia vedy a výskumu na fakulte, ktorá obsahuje tabuľky *Publikacia* (cisloP, nazovP, pocet_stran), *Autor* (cisloA, menoA, ustav) a *Je_autorom* (cisloP, cisloA, podiel). U publikácie je uvedené jej jednoznačné číslo, názov a počet strán, u autora je uvedené jednoznačné číslo, meno a ústav, na ktorom pracuje. V tabuľke *Je_autorom* je informácia o autoroch jednotlivých publikácií, pričom pre každého autora publikácie je uvedený podiel, aký na danej publikácii mal (v percentách). Primárne kľúče sú podčiarknuté. Napíšte v jazyku SQL nasledujúce dotazy/príkazy:

- Na ktorých publikáciách s počtom strán menším než 20 sa podieľali autori z troch a viac rôznych ústavov? (cisloP, nazovP, pocet_autorov)

```

SELECT cisloP, nazovP, COUNT (*) pocet_autorov
FROM Publikacia NATURAL JOIN Je_autorom NATURAL JOIN Autor
WHERE pocet_stran < 20
GROUP BY cisloP, nazovP
HAVING COUNT (DISTINCT ustav) >= 3

```

- Ktorí autori písali **všetky** publikácie sami, tj. majú na **všetkých** svojich publikáciách podiel 100%? (cisloA, menoA)

```

SELECT DISTINCT cisloA, menoA
FROM Autor, Je_autorom J
WHERE A.cisloA = J.cisloA AND podiel = 100
AND NOT EXISTS (
    SELECT * FROM Je_autorom J2
    WHERE A.cisloA = J2.cisloA AND podiel < 100)

```

- Napíšte príkaz pre vytvorenie databázového kurzora, ktorý umožní sprístupniť riadky výsledku dotazu: Na kolkých publikáciách sa podieľali jednotliví autori z ústavu 'UITS'? (cisloA, menoA, pocet_publicacii). Ďalej uveďte, či by bolo možné tento kurzor použiť pre aktualizáciu záznamov databázy a svoje tvrdenie zdôvodnite.

```

DECLARE cursor1 CURSOR FOR
SELECT cisloA, menoA, COUNT (*) pocet_publicacii
FROM Autor NATURAL JOIN Je_autorom
WHERE ustav = 'UITS'
GROUP BY cisloA, menoA

```

- Ktorí autori mimo ústav 'UIFS' sa podieľali na viacerých publikáciách než každý z autorov u 'UIFS'? (cisloA, menoA)

```

SELECT cisloA, menoA
FROM Autor NATURAL JOIN Je_autorom
WHERE ustav <> 'UIFS'
GROUP BY cisloA, menoA
HAVING COUNT (*) > ALL (SELECT COUNT (*)
                        FROM Autor NATURAL JOIN Je_autorom
                        WHERE ustav = 'UIFS'
                        GROUP BY cisloA
)

```

- Napíšte príkaz pre vytvorenie databázového kurzora, ktorý umožní sprístupniť riadky výsledku dotazu: Ktoré publikácie majú viac než troch autorov? (cisloP, nazovP, pocet_autorov). Ďalej uveďte, či by bolo možné tento kurzor použiť pre aktualizáciu záznamov databázy a svoje tvrdenie zdôvodnite.

```

DECLARE cursor1 CURSOR FOR
SELECT cisloP, nazovP, COUNT (*) pocet_autorov
FROM Publikacia NATURAL JOIN Je_autorom
GROUP BY cisloP, nazovP
HAVING COUNT(*) > 3

```

3. Uvažujte časť relačnej databázy banky týkajúcej sa klientov a účtov, ktorá obsahuje tabuľky *Klient* (RC, meno, mesto, rok_nar), *Disponuje* (RC, c_uctu, limit) a *Ucet* (c_uctu, stav, typ, vlastnik). O klientovi ukladáme jeho rodné číslo, meno, mesto bydliska a rok narodenia, o účte jednoznačné číslo účtu, stav, typ (napr. sporiaci, bežný, atď.) a cudzí kľúč do tabuľky *Klient* odkazujúci sa na vlastníka. V tabuľke *Disponuje* sú údaje o ďalších disponentoch (mimo vlastníka) jednotlivých účtov (cudzíe kľúče do tabuliek *Klient* a *Ucet*) a ich limity výberu. Primárne kľúče sú podčiarknuté. Zapište v SQL nasledujúce dotazy/príkazy (zoznam stĺpcov v zátvorke udáva požadovaný tvar výsledkov):

- Ktoré účty, na ktorých je stav vyšší než 100 000 Kč, nemajú žiadneho disponenta? (c_uctu, stav, typ)

```

SELECT c_uctu, stav, typ
FROM Ucet NATURAL LEFT JOIN Disponuje
WHERE stav > 100000 AND RC IS NULL

```

- Ktorí disponenti majú na všetkých účtoch, s ktorými disponujú, limit nižší než 5000 Kč? (RC, meno, mesto)

```

SELECT RC, meno, mesto
FROM Klient K, Disponuje D
WHERE K.RC = D.RC AND limit < 5000
AND NOT EXISTS
    (SELECT * FROM Disponuje D2
     WHERE K.RC = D2.RC

```

AND limit >= 5000)

- Napíšte príkaz pre pridanie nového atribútu `datum_zmeny` do tabulky *Klient*. Ďalej napíšte príkaz pre vytvorenie databázového triggera, ktorý v prípade vloženia záznamu alebo jeho modifikácie uloží do atribútu `datum_zmeny` aktuálny dátum:

```
ALTER TABLE Klient ADD datum_zmeny DATE
```

```
CREATE TRIGGER set_datum
BEFORE INSERT OR UPDATE ON Klient
FOR EACH ROW
BEGIN
    :NEW.datum_zmeny := SYSDATE;
END
```

- Ktorí klienti disponujú s účtami 10 a viac rôznych vlastníkov? (RC, meno, mesto, `pocet_uctov`)

```
SELECT RC, meno, mesto, COUNT (*) pocet_uctov
FROM Klient NATURAL JOIN Disponuje NATURAL JOIN Ucet
GROUP BY RC, meno, mesto
HAVING COUNT (DISTINCT vlastnik) >= 10
```

- S ktorými účtami disponuje najvyšší počet disponentov? Uvažujte prípad, že môže byť viacero účtov s rovnakým maximálnym počtom disponentov (`c_uctu`, `typ`, `pocet_disponentov`)

```
SELECT c_uctu, typ, COUNT (*) pocet_disponentov
FROM Ucet NATURAL JOIN Disponuje
GROUP BY c_uctu, typ
HAVING COUNT (*) >= ALL (
    SELECT COUNT(*)
    FROM Ucet NATURAL JOIN Disponuje
    GROUP BY c_uctu, typ)
```

- Napíšte príkaz pre pridanie nového atribútu `datum_zmeny` do tabulky *Ucet*. Ďalej napíšte príkaz pre vytvorenie databázového triggera, ktorý v prípade vloženia záznamu alebo jeho modifikácie uloží do atribútu `datum_zmeny` aktuálny dátum:

```
ALTER TABLE Ucet ADD datum_zmeny DATE
```

```
CREATE TRIGGER set_datum
BEFORE INSERT OR UPDATE ON Ucet
FOR EACH ROW
BEGIN
    :NEW.datum_zmeny := SYSDATE;
```

END

4. Uvažujte časť relačnej databázy internetového obchodu. Obsahuje tabuľky *Zakaznik* (login, meno, mesto), *Kupil* (login, idzbozi, datum, dodanie) a *Zbozi* (idzbozi, nazov, cena, kategoria). *Login* je jednoznačný login zákazníka, ďalej je u neho uvedené jeho meno a mesto bydliska. U zboží je uvedené jeho jednoznačné číslo (idzbozi), ďalej názov, cena, a kategória, do ktorej spadá. V tabuľke *Kupil* je uvedený okrem loginu zákazníka a identifikátora zboží taktiež dátum objednania a spôsob dodania. Primárne kľúče sú podčiarknuté.

- Zapište v SQL nasledujúci dotaz (zoznam stĺpcov v zátvorke udáva požadovaný tvar výsledku): Ktoré zboží si objednali **len** zákazníci z Brna? (idzbozi, nazov, cena)

```
SELECT DISTINCT idzbozi, nazov, cena
FROM Zbozi NATURAL JOIN Kupil NATURAL JOIN Zakaznik
WHERE mesto = 'Brno' AND idzbozi NOT IN (
    SELECT idzbozi FROM Kupil NATURAL JOIN Zakaznik
    WHERE mesto <> 'Brno'
)
```

- Napíšte výraz relačnej algebry, ktorý odpovedá nasledujúcemu dotazu. Ktoré zboží si objednal zákazník 'Petr Dvořák'? (idbozi, nazov) Pozn.: Ak použijete inú notáciu než bola použitá na prednáškach, je nutné notáciu vysvetliť.

((Zakaznik JOIN Kupil JOIN Zbozi) WHERE meno = 'Petr Dvořák') [idzbozi, nazov]

- Predpokladajte, že do tabuľky *Zbozi* je pridaný atribút *datum_zmeny*. Napíšte príkaz jazyka SQL pre vytvorenie databázového triggeru, ktorý zaistí, že sa pri vložení alebo modifikácii záznamu v tabuľke *Zbozi* u tohto záznamu uloží do atribútu *datum_zmeny* aktuálny dátum.

```
CREATE OR REPLACE TRIGGER nazov
BEFORE INSERT OR UPDATE ON Zbozi
FOR EACH ROW
BEGIN
    :NEW.datum_zmeny := SYSDATE;
END;
```

- Zapište v SQL nasledujúci dotaz (zoznam stĺpcov v zátvorke udáva požadovaný tvar výsledku): Ktoré rôzne zboží majú rovnaký názov, ale spadajú do rôznych kategórií? (nazov, kategoria1, kategoria2)

```
SELECT DISTINCT Z1.nazov, Z1.kategoria, Z2.kategoria
FROM Zbozi Z1, Zbozi Z2
WHERE Z1.nazov = Z2.nazov AND Z1.kategoria > Z2.kategoria
```

- Napište výraz relačnej algebry, ktorý odpovedá nasledujúcemu dotazu: Ktorí zákazníci objednávali zboží 10.5.2017? (login, meno) Pozn.: Ak použijete inú notáciu ako bola použitá na prednáškach, je nutné notáciu vysvetliť.

((Zakaznik JOIN Kupil) WHERE datum='10.5.2017') [login, meno]

- Napište príkaz jazyka SQL pre vytvorenie databázového triggeru, ktorý zaistí, že pri zmazaní záznamu z tabuľky *Zbozi* bude u odpovedajúcich záznamov v tabuľke *Kupil* nastavená hodnota atribútu *idzbozi* na NULL (tj. zaistí udalosť referenčnej akcie ON DELETE SET NULL).

```
CREATE TRIGGER nazov
BEFORE DELETE ON Zbozi
FOR EACH ROW
BEGIN
    UPDATE Kupil SET idzbozi = NULL
    WHERE idzbozi=:OLD.idzbozi
END
```

5. Uvažujte časť relačnej databázy knižnice s tabuľkami *Citatel* (cisloC, meno, rok_nar, mesto), *Vypozička* (cisloC, cisloT, datum_od, datum_do) a *Titul* (cisloT, autor, nazov, zaner). *CisloC* a *CisloT* sú jednoznačné čísla čitateľa, resp. titulu knihy. O čitateľovi ďalej ukladáme jeho meno, rok narodenia a mesto, kde býva, o titule ukladáme jeho autora (predpokladajte pre jednoduchosť, že vždy je len jeden), názov a žáner (predpokladajte tiež len jeden). V tabuľke *Vypozička* je informácia o jednej výpožičke jedného konkrétneho titulu knihy, vrátane dátumu vypožičania a vrátenia. Primárne kľúče sú podčiarknuté. Zapište v SQL nasledujúce dotazy (zoznam stĺpcov v zátvorke udáva požadovaný tvar výsledku):

- Ktoré tituly boli vypožičané behom roku 2017 menej než trikrát? (cisloT, autor, nazov)

```
SELECT cisloT, autor, nazov
FROM Titul NATURAL LEFT JOIN Vypozička
WHERE datum_od BETWEEN '2017-01-01' AND '2017-12-31'
GROUP BY cisloT, autor, nazov
HAVING COUNT(*) <3
```

- Ktoré tituly si požičiavajú **len** čitatelia narodení po roku 2000? (cisloT, autor, nazov)

```
SELECT DISTINCT cisloT, autor, nazov
FROM Titul NATURAL JOIN Vypozička NATURAL JOIN Citatel
WHERE rok_nar > 2000 AND cisloT NOT IN (
    SELECT cisloT
    FROM Vypozička NATURAL JOIN Citatel
    WHERE rok_nar <= 2000)
```

- Napište príkaz jazyka SQL pre deklaráciu databázového kurzora na prechádzanie výsledku dotazu: Koľko rôznych titulov si vypožičali jednotliví čitatelia z Brna (cisloC, meno, pocet_titulov)? Ďalej napíšte príkazy pre prevedenie tohto dotazu a sprístupnenie prvého riadku výsledku. (mená premenných si zvolíte)

```
DECLARE meno CURSOR FOR
SELECT cisloC, meno, COUNT (DISTINCT cisloT) AS pocet_titulov
FROM Citatel NATURAL JOIN Vypozicka
WHERE mesto = 'Brno'
GROUP BY cisloC, meno;
```

```
OPEN meno;
FETCH meno INTO :cisloC, :meno, :pocet;
```

6. Uvažujte časť relačnej databázy pre evidenciu motorových vozidiel a ich majiteľov s tabuľkami *Majitel* (RC, meno, mesto, vek), *Registracia* (RC, VIN, datum_pri, datum_odh) a *Vozidlo* (VIN, znacka, model, rok_vyroby). O majiteľovi ukladáme jeho rodné číslo, meno, mesto bydliska a vek, o vozidle jednoznačné číslo VIN, značku, model a rok výroby. V tabuľke Registracia sú jednotlivé registrácie vozidiel pri zmene majiteľa. Primárnym kľúčom je dvojica (RC, VIN), ktorého zložky sú cudzie kľúče do tabuliek Majitel (RC) a Vozidlo (VIN), dátum prihlásenia a dátum odhlásenia, ktorý je prázdny v prípade, že je vozidlo aktuálne prihlásené pod danú ŠPZ. Primárne kľúče tabuliek sú podčiarknuté. Napište v SQL nasledujúce dotazy:

- Ktoré vozidlo bolo v databáze zaregistrované najskôr, tj. má uvedený najskorší dátum prihlásenia? (VIN, znacka, model, rok_vyroby)

```
SELECT VIN, znacka, model, rok_vyroby
FROM Vozidlo NATURAL JOIN Registracia
WHERE datum_pri = (SELECT MIN(datum_pri) FROM Registracia)
```

- Ktorí majitelia vlastnia **len** vozidlá značky Škoda? (RC, meno, vek)

```
SELECT DISTINCT RC, meno, vek
FROM Vozidlo NATURAL JOIN Registracia NATURAL JOIN Majitel
WHERE znacka = 'Škoda' AND RC NOT IN
    (SELECT RC
     FROM Vozidlo NATURAL JOIN Registracia
     WHERE znacka <> 'Škoda')
```

- Napište príkaz pre vytvorenie databázového pohľadu obsahujúceho registrácie prevedené v roku 2017 (VIN, znacka, model, datum_pri). Ďalej uveďte, či je tento pohľad aktualizovateľný a svoje tvrdenie zdôvodnite.

```
CREATE VIEW pohlad AS
SELECT VIN, znacka, model, datum_pri
FROM Vozidlo NATURAL JOIN Registracia
```


WHERE datum_pri BETWEEN '2017-01-01' AND '2017-12-31'

7. Predpokladajte časť relačnej databázy hudobného vydavateľstva, ktorá obsahuje tabuľky *Skladba* (cisloS, nazovS, dlzka), *Album* (cisloA, nazovA, interpret, typ) a *Obsahuje* (cisloS, cisloA, poradie). Pri skladbe je uvedené jej jednoznačné číslo, názov a dĺžka, pri albume je uvedené jednoznačné číslo, názov, interpret a typ albumu (napr. štúdiové, výber, koncert atď.) V tabuľke Obsahuje je informácia o tom, ktoré skladby sú umiestnené na jednotlivých albumoch, vrátane poradia skladby na danom albume. Primárne kľúče sú podčiarknuté. Napíšte v jazyku SQL nasledujúce dotazy/priказы:

- Ktorá skladba je posledná na albume, ktorý celkom obsahuje najviac skladieb? (cisloS, nazovS, nazovA)

```
SELECT cisloS, nazovS, nazovA
FROM Skladba NATURAL JOIN Obsahuje NATURAL JOIN Album
WHERE poradie = (SELECT MAX(poradie) FROM Obsahuje)
```

- Ktoré skladby (cisloS, nazovS) sú len na štúdiových albumoch? (tj. len na albumoch typu "studio" a žiadnych iných)

```
SELECT cisloS, nazovS, nazovA
FROM Skladba S, Obsahuje O, Album A
WHERE S.cisloS = O.cisloS AND O.cisloA = A.cisloA AND typ = 'studio' AND NOT
EXISTS (SELECT * FROM Obsahuje O1 and Album A1
        WHERE S.cisloS = O1.cisloS AND O1.cisloA = A1.cisloA AND typ <> 'studio')
```

- Napíšte príkaz pre vytvorenie databázového pohľadu, ktorý bude zobrazovať, na koľkých rôznych albumoch sa nachádzajú jednotlivé skladby. (cisloS, nazovS, pocet_albumov) Uvedte, či je daný pohľad aktualizovateľný a svoje tvrdenie zdôvodnite.

```
CREATE VIEW pohlad (cisloS, nazovS, pocet_albumov) AS
SELECT cisloS, nazovS, COUNT (*)
FROM Skladba NATURAL JOIN Obsahuje
GROUP BY cisloS, nazovS
```

- Ktoré rovnako pomenované albumy nahrali dvaja rôzni interpreti? (nazovA, interpret1, interpret2)

```
SELECT A1.nazovA, A1.interpret, A2.interpret
FROM Album A1, Album A2
WHERE A1.nazovA = A2.nazovA AND A1.interpret > A2.interpret
```

- Ktoré albumy obsahujú len skladby kratšie než 4 minúty? (cisloA, nazovA, interpret)

```
SELECT cisloA, nazovA, interpret
FROM Album A
```

WHERE NOT EXISTS

(SELECT * FROM Skladba S, Obsahuje O

WHERE S.cisloS = O.cisloS AND O.cisloA = A.cisloA AND dlzka >= 4)